

Understanding CMOS Logic through Transistor-Level Simulation

EXPLORE MORE PROJECTS

Learning & Simulation Report Prepared By:

Devansh Umar (5th Sem)

For the coursework – **CMOS Digital & VLSI Design**



DEVANSH UMAR

Electrical & Instrumentation Eng. Student

Key Points of the Simulation –

Why I Built This	2
Introduction to CMOS	2
CMOS Design Principles.....	3
☐ Pull-Up Network (PUN)	3
☐ Pull-Down Network (PDN)	3
☐ Complementary Operation.....	3
☐ Series and Parallel Connections.....	3
☐ Importance of De Morgan's Theorem	4
LTspice & Simulation Methodology.....	4
CMOS NOT Gate.....	5
CMOS NAND Gate	7
CMOS NOR Gate.....	9
CMOS AND Gate.....	11
CMOS OR Gate	13
CMOS XNOR Gate.....	15
CMOS XOR Gate.....	17
Quick Comparison of my all Implemented CMOS Gates	19
Challenges Faced	19
What I Learned	20
References.....	20

Why I Built This

During the beginning of this semester, while studying **CMOS Digital & VLSI Design**, I realized that I was learning Boolean expressions, truth tables and CMOS circuits mostly from theory. I could understand how a logic gate should behave, but I was still curious about one thing:

“How a simple logic gate is actually created using transistors?”

That question became the reason behind this project & also my teacher instructed me to do this.

Instead of only remembering that NAND gives LOW for 11 or XOR becomes HIGH when both inputs are different, I wanted to understand how PMOS and NMOS transistors are arranged so that these logic functions are produced automatically.

Initially, I started **discrete simulation** because its representation looked very similar to the circuits we usually draw in our notebooks. It helped me understand the basic CMOS inverter, NAND and NOR gates. However, when I started combining transistor networks to build more complex gates such as AND and OR, the simulation was not behaving as expected. So, I shifted back to my LTspice, where transistor-level behaviour could be verified more accurately through transient analysis.

I prepared this report not for the practical manual or a research paper. It is simply a record of what I learned while exploring CMOS logic gates at the transistor level. I just wanted to understand the concepts properly instead of memorizing them, and hopefully these notes can also help someone who is beginning to learn CMOS Digital & VLSI Design.

Introduction to CMOS

Digital systems are built using logic gates such as NOT, NAND, NOR, AND, OR, XOR and XNOR. Although these gates are usually represented by simple symbols, every gate inside an integrated circuit is actually formed by connecting transistors in a specific arrangement.

One of the most widely used technologies for implementing digital circuits is **CMOS (Complementary Metal-Oxide-Semiconductor)**. The word complementary comes from the fact that CMOS uses two different types of MOSFETs together:

- PMOS (P-channel MOSFET)
- NMOS (N-channel MOSFET)

Instead of working independently, these two transistor networks complement each other. One network connects the output to the supply voltage (VDD), while the other connects it to ground (GND). Because only one of these networks conducts under steady-state conditions, CMOS circuits consume **very little static power** while still producing reliable logic levels.

Today, CMOS technology is used in almost every modern digital electronic system including microprocessors, microcontrollers, memories and VLSI chips. Understanding how these basic CMOS gates are designed forms the foundation for studying larger digital circuits later.

CMOS Design Principles

While working on these gates, I found that every CMOS circuit follows a common design principles. Once these principles become clear, designing different logic gates becomes much easier than trying to memorize individual transistor connections.

➤ Pull-Up Network (PUN)

Pull-Up Network is built using **PMOS** transistors and is connected between the output node and the positive supply voltage (VDD). Whenever the required input condition is satisfied, this network conducts and pulls the output to logic **HIGH**.

➤ Pull-Down Network (PDN)

Pull-Down Network is built using **NMOS** transistors and is connected between the output node and ground (GND). Under the appropriate input conditions, this network conducts and pulls the output to logic **LOW**.

➤ Complementary Operation

One of the most interesting observations was that PMOS and NMOS networks complement each other. When the Pull-Up Network conducts, the Pull-Down Network remains OFF, and when the Pull-Down Network conducts, the Pull-Up Network remains OFF. This complementary behaviour is the reason why CMOS circuits consume very little power during steady-state operation.

➤ Series and Parallel Connections

While designing the transistor networks, another useful pattern became clear.

For the NMOS Pull-Down Network:

- Series connection represents an AND condition.
- Parallel connection represents an OR condition.

The PMOS Pull-Up Network is obtained by creating the **dual of the NMOS network**. In simple words, every series connection is replaced by a parallel connection and every parallel connection is replaced by a series connection while keeping the same input variables. This observation became very useful while designing NAND, NOR, XOR and XNOR gates.

➤ Importance of De Morgan's Theorem

Another concept that repeatedly appeared during this project was De Morgan's theorem. Instead of being just a Boolean algebra identity, it directly explains why PMOS and NMOS networks are complementary. Once I started applying De Morgan's theorem while designing the transistor networks, the CMOS gate structures became much easier to understand.

LTspice & Simulation Methodology

I used LTspice to verify every CMOS gate because unlike digital logic simulators, LTspice models the electrical behaviour of transistors, which helped me to observe the output waveform for the different-different input conditions.

To activate the circuit I used the 5V DC supply as VDD for complementary PMOS and NMOS transistor networks & for the Gate supply voltage, I used PULSE voltage source to get 00 01 10 11 combination as output result.

For two-input gates, I selected the pulse timings as –

Time Interval	Input (A)	Input (B)
0 - 1 ms	0	0
1 - 2 ms	0	1
2 - 3 ms	1	0
3 - 4 ms	1	1

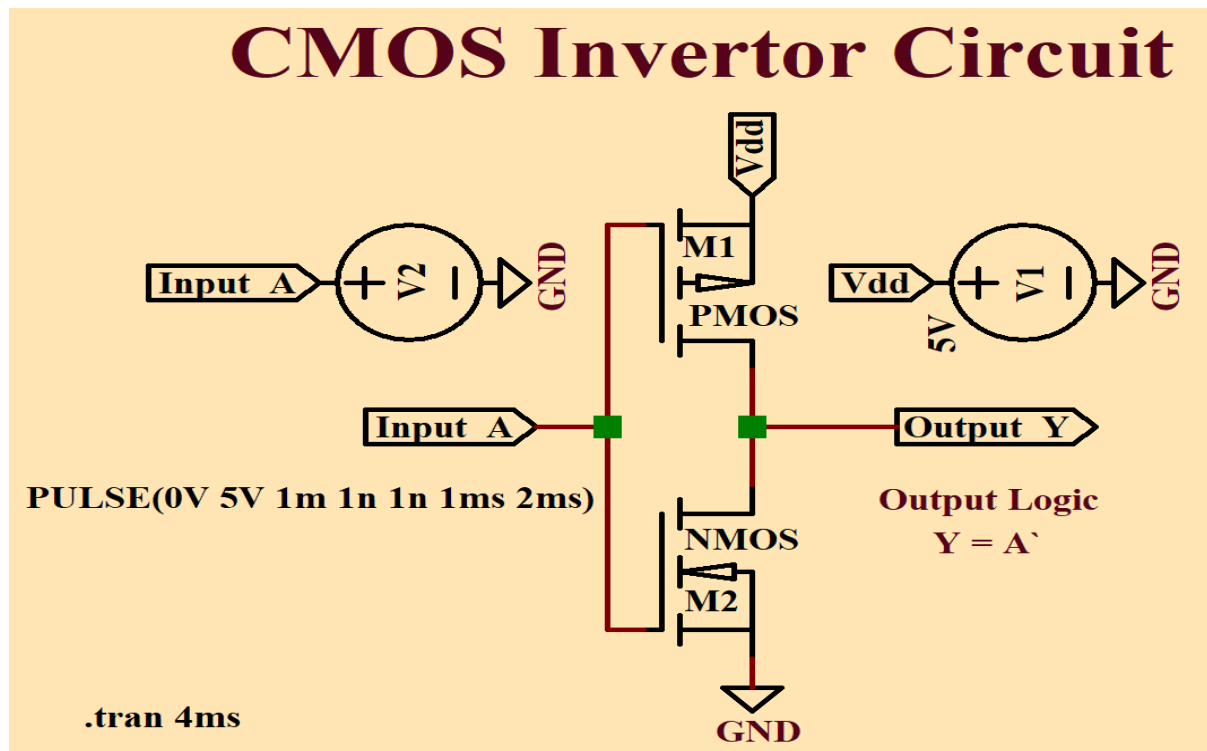
This sequence made it easier to compare the simulated output directly with the truth table of each gate.

CMOS NOT Gate

Boolean Expression

$$Y = A'$$

LTspice schematic



Output Waveform



Truth Table

Input (A)	Output [Y = A']
0	1
1	0

Working Principle

- NOT gate is the simplest CMOS logic gate and also the first circuit that helped me understand how PMOS and NMOS transistors work together.
- The circuit consists of one PMOS transistor connected to VDD and one NMOS transistor connected to GND. Both transistor gates are connected to the same input.
- When the input is LOW, the PMOS transistor turns ON while the NMOS transistor remains OFF. As a result, the output is connected to VDD and becomes logic HIGH.
- When the input becomes HIGH, the PMOS transistor turns OFF and the NMOS transistor turns ON. This connects the output directly to ground, producing logic LOW.

Since only one transistor conducts at a time, the output always becomes the opposite of the input.

Quick Remember

- NOT gate simply changes every input to its opposite.
- 0 becomes 1 and 1 becomes 0.

Observation

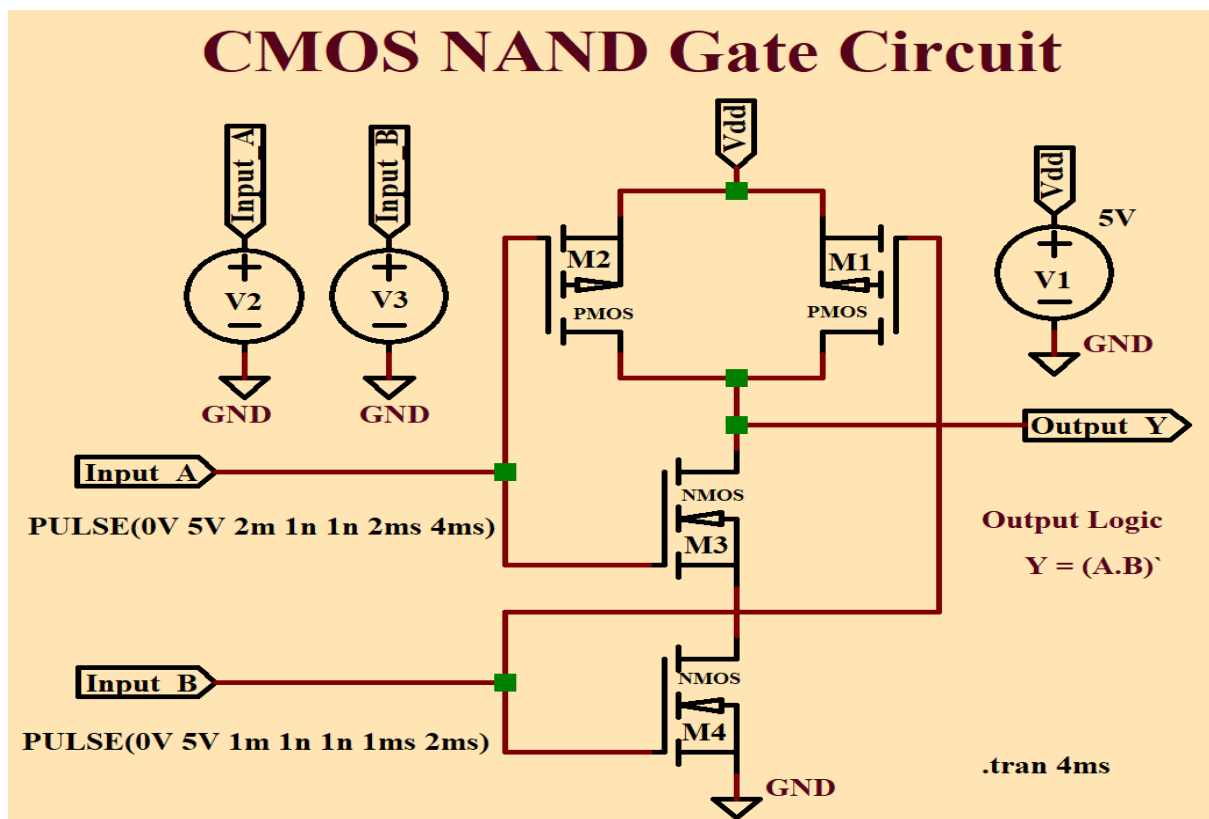
In the simulated waveform we can see that the output always remains opposite to the input throughout the transient analysis. This confirms the correct operation of the CMOS inverter.

CMOS NAND Gate

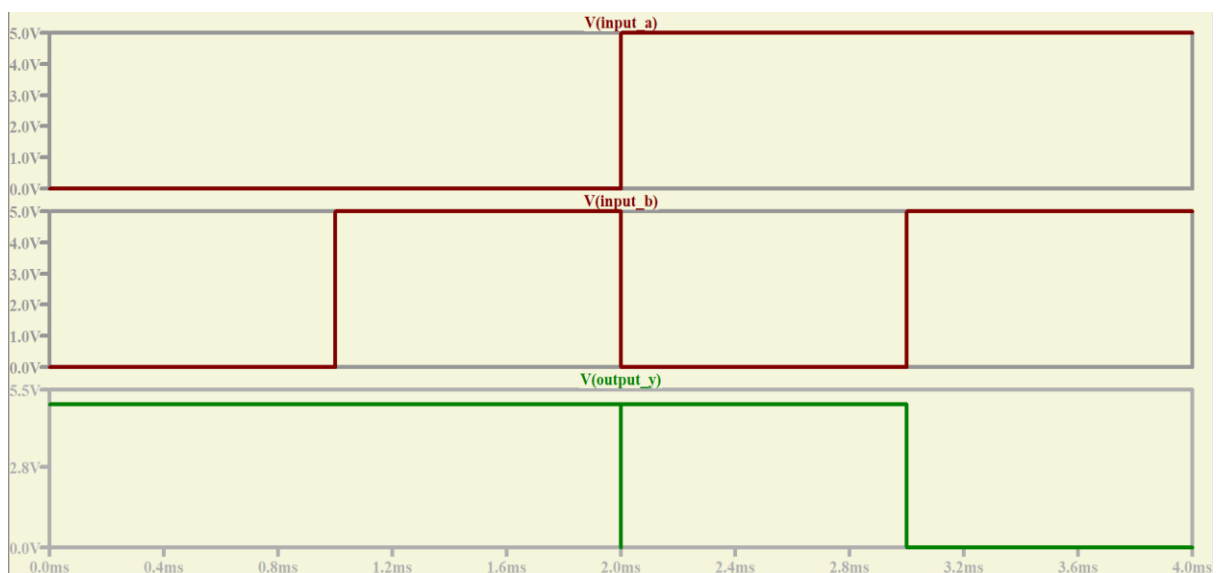
Boolean Expression

$$Y = (A.B)'$$

LTspice schematic



Output Waveform



Truth Table

Input (A)	Input (B)	Output [Y = (A.B)']
0	0	1
0	1	1
1	0	1
1	1	0

Working Principle

- NAND gate was the first two-input CMOS gate that made the relationship between Boolean expressions and transistor connections much clearer.
- In this Pull-Up Network consists of two PMOS transistors connected in parallel, while the Pull-Down Network consists of two NMOS transistors connected in series.
- If either input becomes LOW, at least one PMOS transistor conducts and connects the output to VDD. However, both NMOS transistors conduct only when both inputs are HIGH.

Therefore, the output remains HIGH for three input combinations and becomes LOW only when both inputs are HIGH.

While building this gate, I also understood why NAND is considered a universal gate. More complex gates such as AND can easily be obtained by adding a CMOS inverter to its output.

Quick Remember

- NAND only says "No" when everyone says "Yes".
- The output becomes LOW only for 11.

Observation

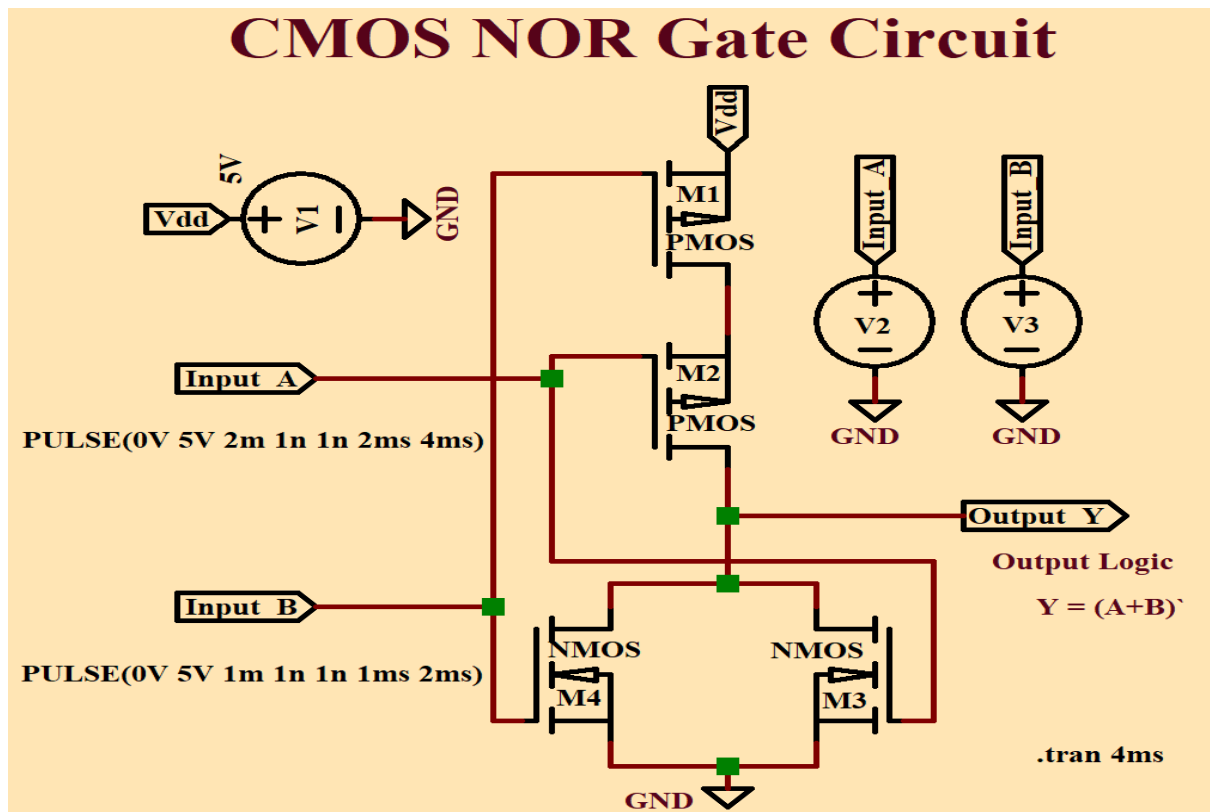
I found that the output stays HIGH except when both inputs become HIGH simultaneously which is exactly what truth table says.

CMOS NOR Gate

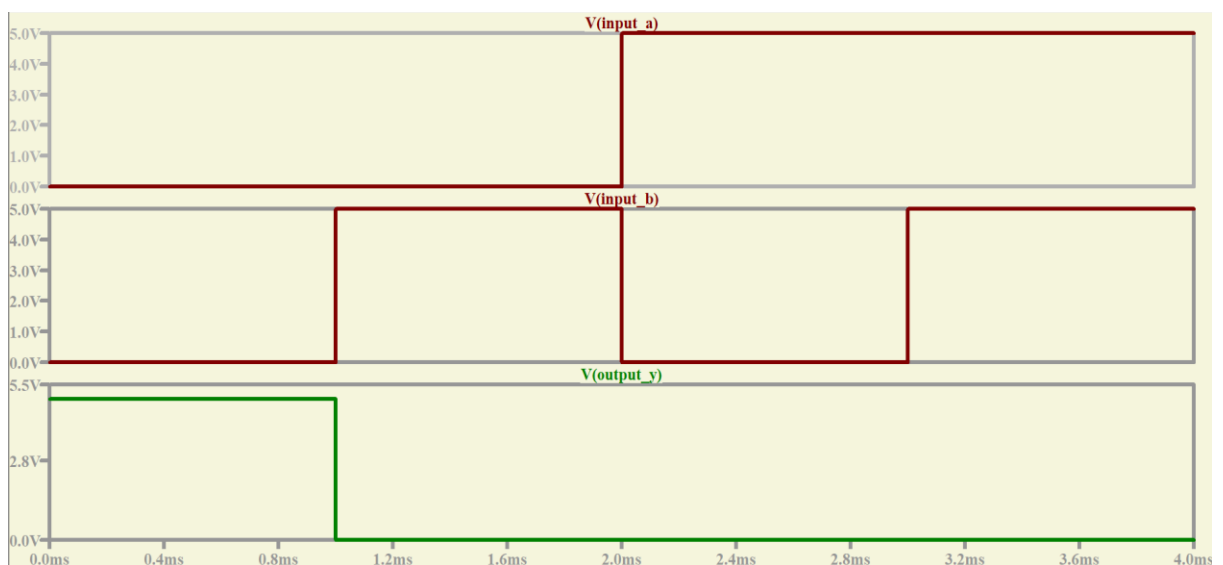
Boolean Expression

$$Y = (A+B)'$$

LTspice schematic



Output Waveform



Truth Table

Input (A)	Input (B)	Output [$Y = (A + B)'$]
0	0	1
0	1	0
1	0	0
1	1	0

Working Principle

- NOR gate follows the opposite transistor arrangement compared to the NAND gate.
- In this circuit, the PMOS transistors are connected in series, while the NMOS transistors are connected in parallel.
- Whenever either input becomes HIGH, at least one NMOS transistor provides a path to ground and the output becomes LOW.
- For the output to become HIGH, both NMOS transistors must remain OFF. This happens only when both inputs are LOW, allowing both PMOS transistors to conduct and connect the output to VDD.

Quick Remember

- NOR says "Yes" only when everyone says "No".
- The output becomes HIGH only for 00.

Observation

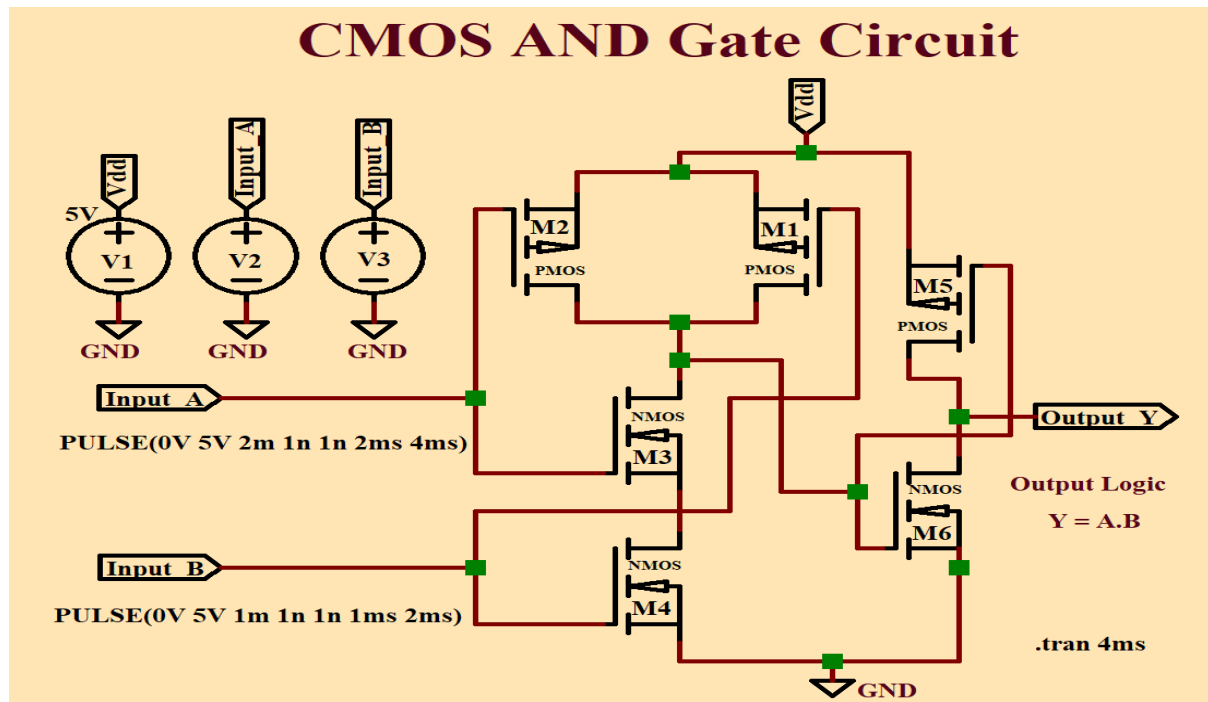
LTspice waveform confirms that the output becomes HIGH only when both inputs are LOW while every other input combination produces a LOW output.

CMOS AND Gate

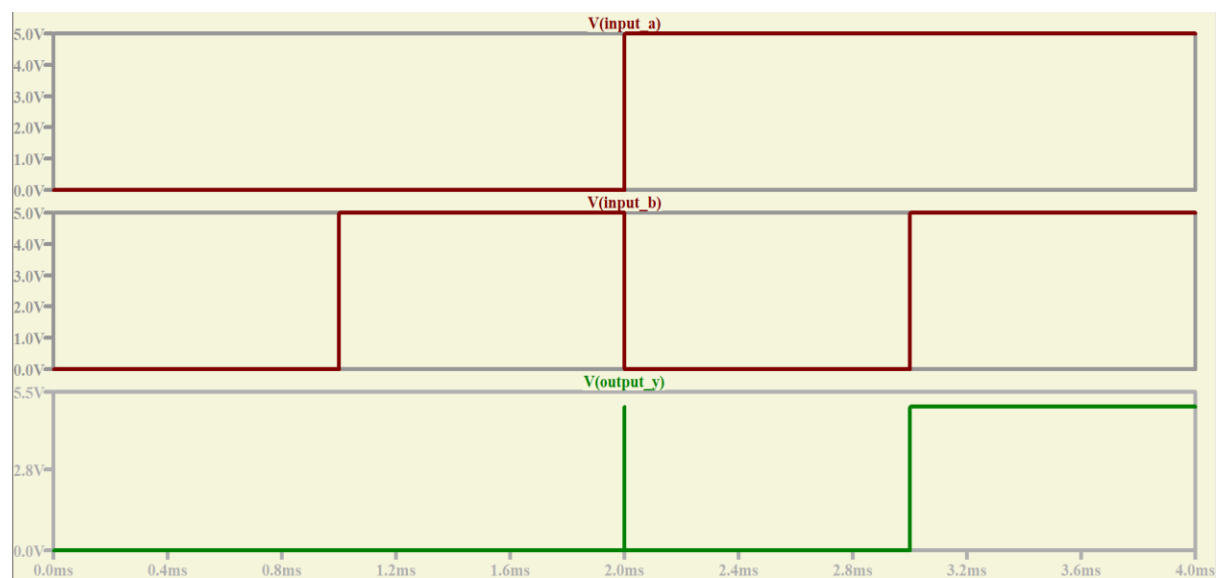
Boolean Expression

$$Y = A.B$$

LTspice schematic



Output Waveform



Truth Table

Input (A)	Input (B)	Output [Y = A.B]
0	0	0
0	1	0
1	0	0
1	1	1

Working Principle

- Instead of designing the AND gate directly from scratch, I built it by connecting a CMOS inverter to the output of the NAND gate.
- The NAND section first produces the complement of the AND operation. The additional CMOS inverter then reverses that output to obtain the required logic.
- This approach helped me understand how universal gates can be used to construct other logic functions instead of designing every gate independently.
- The output becomes HIGH only when both inputs are HIGH. In every other case, at least one transistor arrangement inside the NAND section prevents the required output condition, and the inverter produces the final AND output.

Quick Remember

- AND needs everyone's permission.
- Unless every input is HIGH, the output remains LOW.

Observation

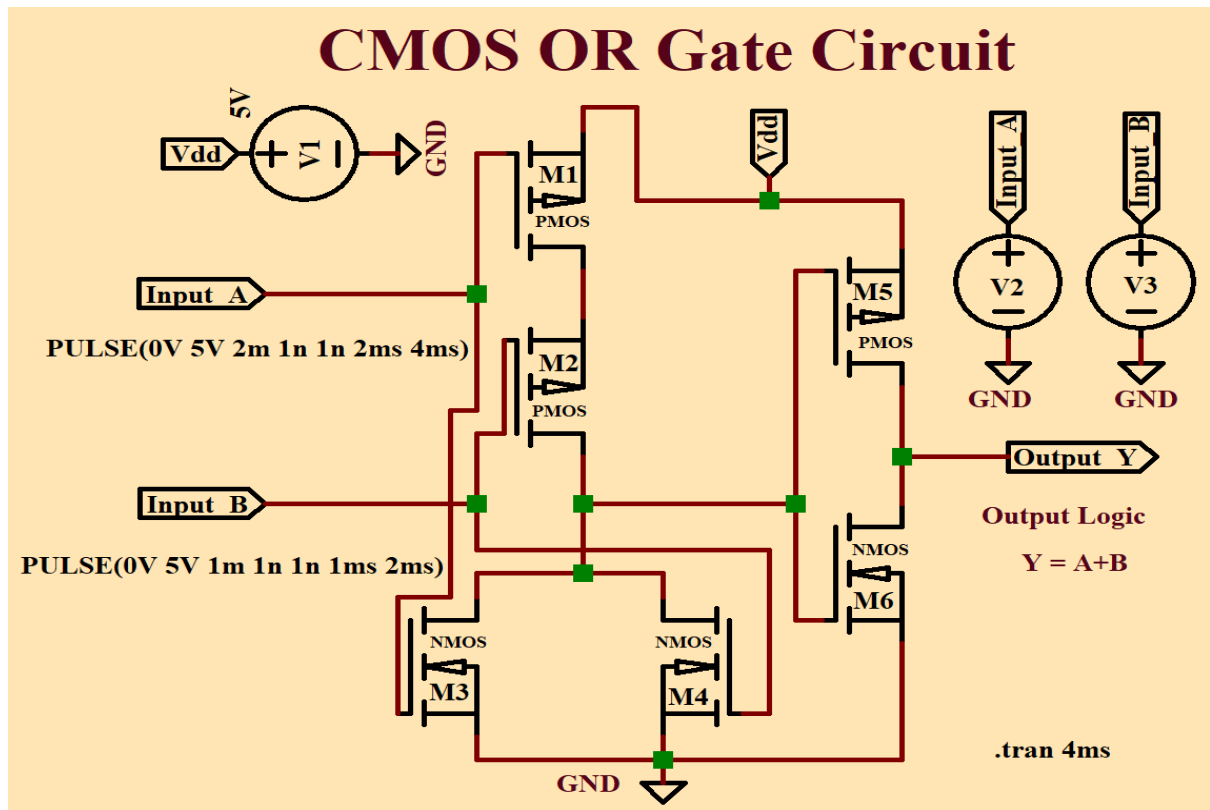
The output waveform remains LOW for the first three input combinations and changes to HIGH only when both inputs become HIGH, matching the expected AND truth table.

CMOS OR Gate

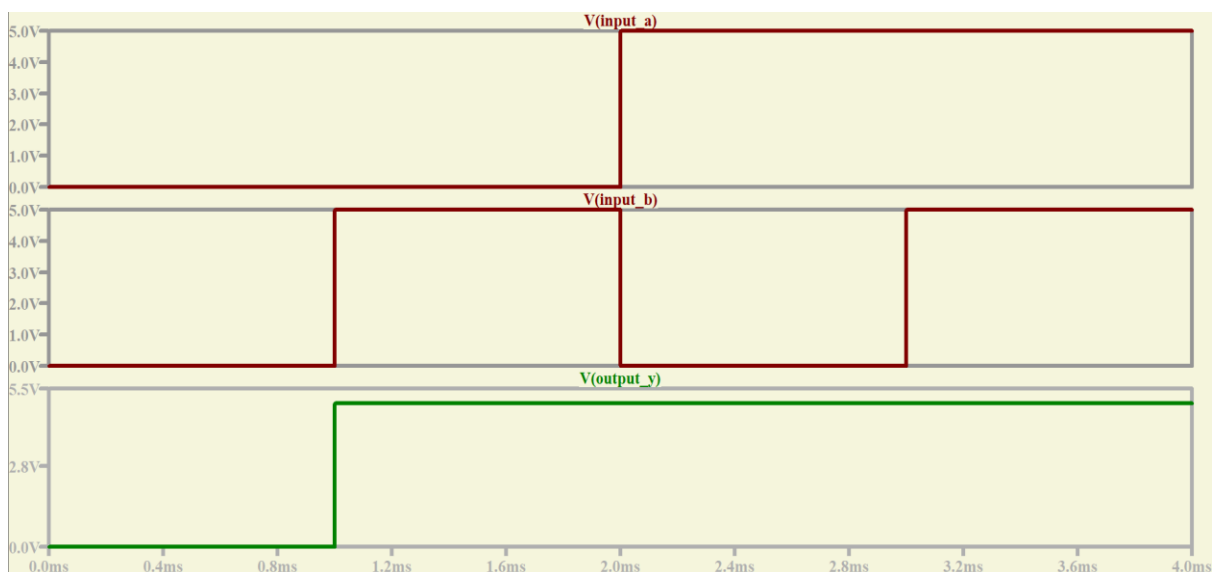
Boolean Expression

$$Y = A + B$$

LTspice schematic



Output Waveform



Truth Table

Input (A)	Input (B)	Output [Y = A + B]
0	0	0
0	1	1
1	0	1
1	1	1

Working Principle

- Instead of designing a completely new circuit, I simply placed a CMOS inverter to the output of a NOR gate. This helped me to understand, how universal gates can be converted into another logic function by simply adding an inverter & also saved my effort.
- NOR Gate first generates the complemented output, and the inverter reverses it to obtain the OR Gate logic.
- One thing I noticed while simulating this circuit was that the waveform immediately became easier to predict after understanding the NOR gate, as it is simply complemented form of NOR Gate.

Quick Remember

- Even one HIGH is enough.
- If any input becomes HIGH, the output also becomes HIGH.

Observation

Simulation showed that the output remains LOW only when both inputs are LOW. Every other input combination produces a HIGH output, which is same as in OR Gate truth table.

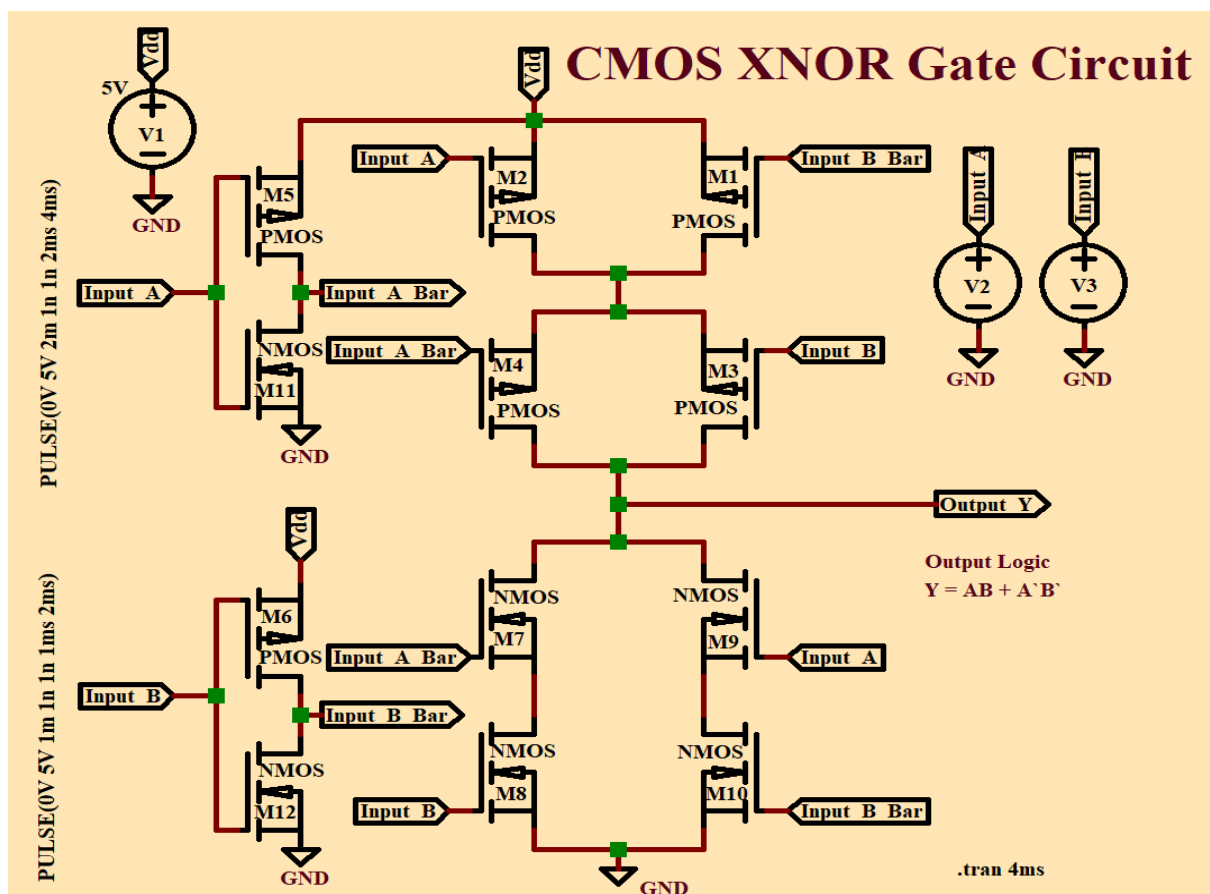
CMOS XNOR Gate

Boolean Expression

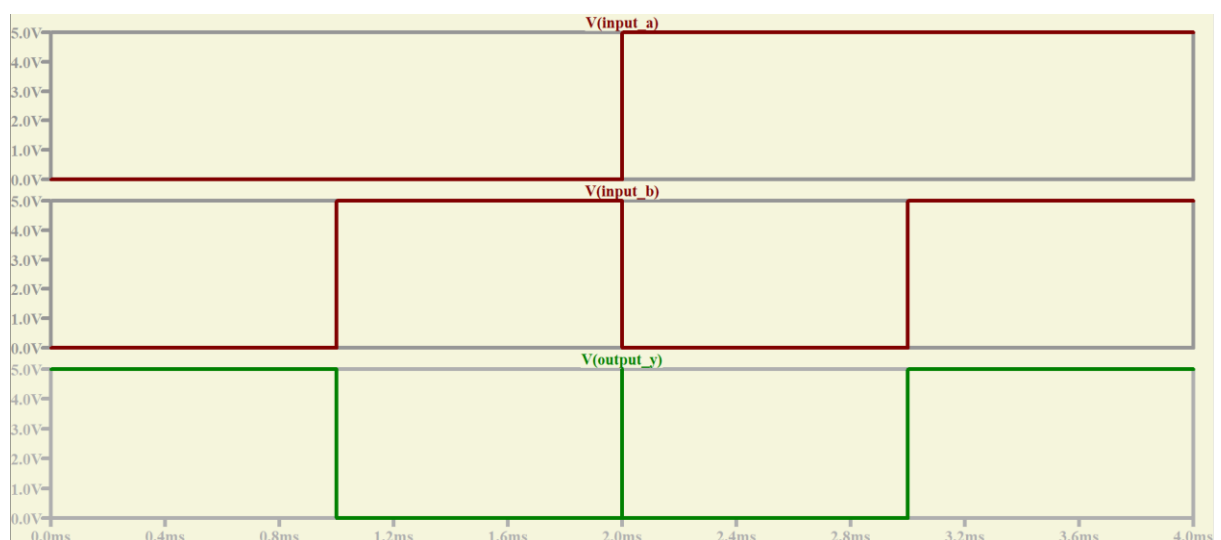
$$Y = AB + A'B'$$

$$\text{Or } Y = A \oplus B$$

LTspice schematic



Output Waveform



Truth Table

Input (A)	Input (B)	Output [$Y = A \oplus B$]
0	0	1
0	1	0
1	0	0
1	1	1

Working Principle

- Among all the gates in this project, XNOR took the longest time to understand.
- Unlike NAND or NOR, the transistor arrangement cannot be obtained by simply placing PMOS and NMOS transistors in basic series or parallel combinations. Additional complementary input signals, $\bar{A}$ and $\bar{B}$, are required before building the pull-up and pull-down networks.
- While designing this gate, I spent more time understanding why each transistor belongs to a particular branch rather than copying a circuit diagram. Once the relationship between the Boolean expression and the complementary CMOS networks became clear, the overall structure started making sense.

This gate also helped me understand why XOR & XNOR is considered one of the more complex basic logic gates in CMOS design.

Quick Remember

- XNOR looks for similarity.
- If both inputs are the same, the output becomes HIGH.

Observation

In the output waveform I saw that the output becomes HIGH only for 00 and 11 combination, confirming the correct implementation of the CMOS XNOR gate.

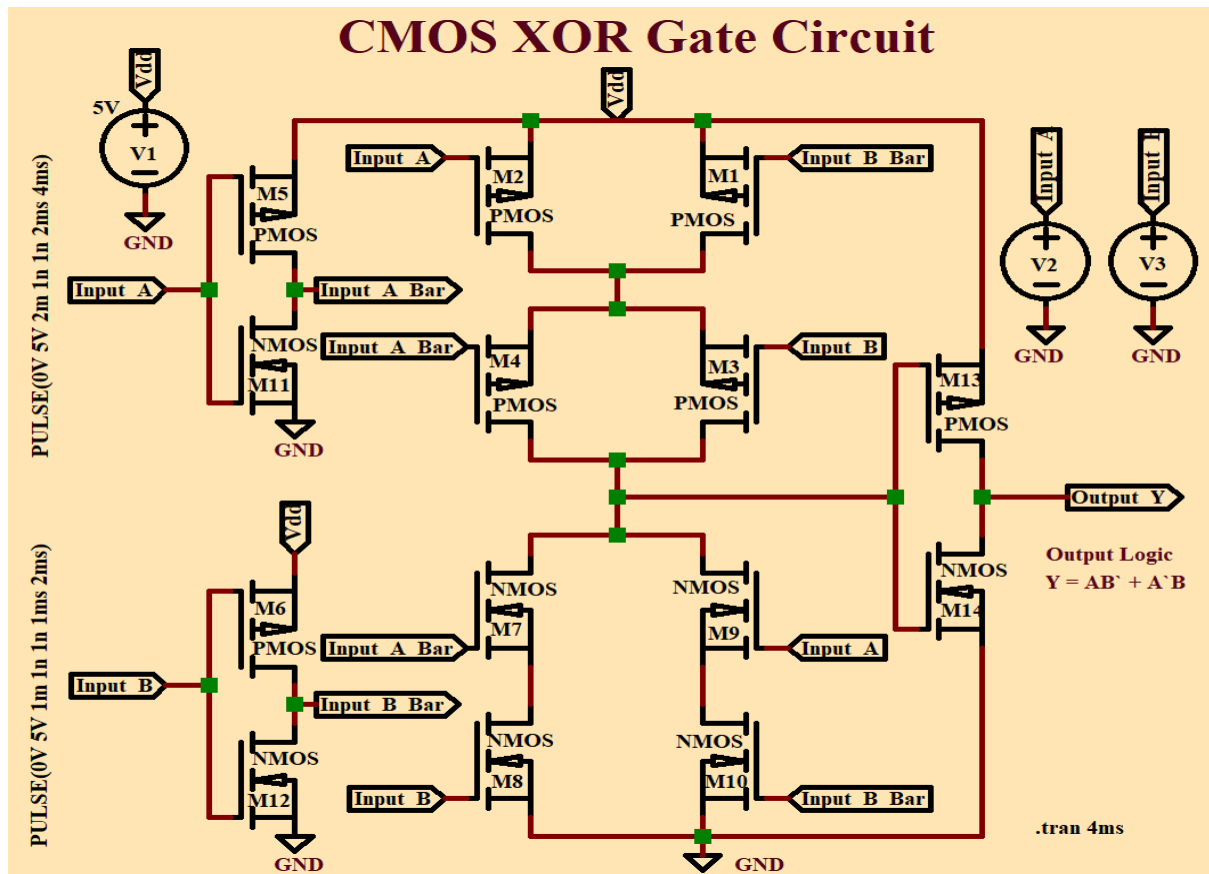
CMOS XOR Gate

Boolean Expression

$$Y = AB' + A'B$$

$$\text{Or } Y = A \otimes B$$

LTspice schematic



Output Waveform



Truth Table

Input (A)	Input (B)	Output [$Y = A \otimes B$]
0	0	0
0	1	1
1	0	1
1	1	0

Working Principle

- The XOR gate became much easier to build after completing the XNOR gate because both gates are closely related.
- Instead of starting from scratch, I just placed a CMOS Inverter circuit at the output of the XNOR to get the complement & I got the XOR gate behaviour easily.
- During this part of the project, I realized that CMOS gate design is much more about understanding transistor networks than remembering final circuits. Once the Boolean expression and the duality principle become clear, the transistor arrangement can be derived step by step.

Although this gate uses more transistors than the previous gates, the design follows the same CMOS principles that were already learned while building NAND and NOR.

Quick Remember

- XOR looks for differences.
- If the inputs are different, the output becomes HIGH.

Observation

In the output waveform we can see that the output becomes HIGH only for the input combinations 01 and 10, confirming the correct XOR operation.

Quick Comparison of my all Implemented CMOS Gates

Gate	Boolean Expression	Output becomes High when...
NOT	A'	Input is LOW
NAND	$(A.B)'$	At least one input is LOW
NOR	$(A + B)'$	Both inputs are LOW
AND	$A.B$	Both inputs are HIGH
OR	$A + B$	Any input is HIGH
XNOR	$AB + A'B' \text{ Or } Y = A \oplus B$	Inputs are different
XOR	$AB' + A'B \text{ Or } Y = A \otimes B$	Inputs are the same

Challenges Faced

No project is completed without running into a few problems, and this one was no exception.

Initially, I started building the transistor-level circuits in **Logisim Evolution** because it provides a clean visual representation of transistors. It worked well for simple CMOS gates such as **NOT**, **NAND** and **NOR**.

However, while trying to build larger circuits like **AND** and **OR** by combining transistor networks, the simulation behaviour was weird. It started showing some random spikes at some places & sometimes flattens not working properly for **combination of circuits**. After spending some time troubleshooting, I decided to move back to my LTspice, for the reliable results.

Another challenge was XOR and XNOR gates, it took my whole time I've to give up & restart on the next day. Correct wiring with visually clear schematic was dam tough. XNOR was the most **complicated** and **time taking** one as for XOR I just placed a CMOS Invertor at the output.

Looking back, these challenges were probably the most valuable part of the simulations because they improved my understanding much more than simply obtaining the correct output waveform.

What I Learned

When I started this project, CMOS logic gates were simply symbols and truth tables that I had studied in class. After designing each gate transistor by transistor, I started looking at them differently.

Instead of remembering truth tables, I now try to think about which transistor network should conduct for a particular input condition. This approach makes the Boolean expression, transistor arrangement and output waveform feel connected rather than independent topics.

One of the most interesting lessons was realizing that CMOS design follows a logical pattern. NAND, NOR, AND and OR can be derived systematically, while even the more complex XOR and XNOR gates become manageable once complementary transistor networks are understood.

References

Online Resources and my **college notebook & lectures of CMOS Digital & VLSI Design** which I studied from my teacher **Dr. YP Pundir Sir**.